

TanStack React Query — Guía rápida

Server state · cache · sync · v5 · Entrevistas front React

¿QUÉ ES Y CUÁNDO USARLO?

React Query gestiona *server state*: datos que viven en el API, no en React. Cache, refetch, loading/error, deduplicación y sincronización entre componentes.

Usar Query	NO usar Query (usar useState/Context)
GET listados, detalle, búsquedas	Tema UI, modal abierto, step wizard
POST/PUT/DELETE al servidor	Formulario antes de submit
Datos compartidos entre pantallas	Auth token en Context (ok)

SETUP MÍNIMO

```
npm i @tanstack/react-query

// app/providers.tsx
'use client'
import { QueryClient, QueryClientProvider } from
 '@tanstack/react-query'
import { useState } from 'react'

export function Providers({ children }) {
  const [client] = useState(() => new QueryClient({
    defaultOptions: {
      queries: {
        staleTime: 60_000,      // 1 min fresh
        retry: 1,
        refetchOnWindowFocus: false,
      },
    },
  }))
  return (
    <QueryClientProvider client={client}>
      {children}
    </QueryClientProvider>
  )
}
```

CONCEPTOS CLAVE

Concepto	Qué hace
queryKey	ID único del cache. Array jerárquico: ['policies', id]
queryFn	Función async que devuelve datos
staleTime	Tiempo “fresco” — no refetch automático
gcTime	Tiempo en cache tras quedar unused (antes cacheTime)
enabled	false = no ejecutar hasta condición
select	Transformar datos antes de render
placeholderData	UI optimista mientras carga

USEQUERY — LEER DATOS

```
function PolicyDetail({ id }: { id: string }) {
  const { data, isLoading, isError, error, isFetching, refetch } = useQuery({
    queryKey: ['policy', id],
    queryFn: () => fetch(`/api/policies/${id}`).then(r => {
      if (!r.ok) throw new Error('Failed')
      return r.json()
    }),
    staleTime: 5 * 60_000,
    enabled: !!id,
  })

  if (isLoading) return <p>Loading...</p>
  if (isError) return <p>Error: {error.message}</p>
  return (
    <div>
      {isFetching && <span>Updating...</span>}
      <h1>{data.name}</h1>
      <button onClick={() => refetch()}>Refresh</button>
    </div>
  )
}
```

Estados útiles

Flag	Significado
isLoading	Primera carga sin cache
isFetching	Cualquier fetch en curso (incl. background)
isPending	v5: sin data aún
isSuccess / isError	Terminal states
status	'pending' 'error' 'success'

QUERYKEY — JERARQUÍA

```
['policies'] // lista
['policies', { status: 'open' }] // lista filtrada
['policies', policyId] // detalle

// Invalidar TODO policies:
invalidateQueries({ queryKey: ['policies'] })
// También invalida ['policies', id] hijos
```

Las keys son **serializables** (JSON). Objetos en key = refetch al cambiar filtro.

USEMUTATION — ESCRIBIR

```
const queryClient = useQueryClient()

const mutation = useMutation({
  mutationFn: (body: PolicyInput) =>
    fetch('/api/policies', {
      method: 'POST',
      body: JSON.stringify(body),
    }).then(r => r.json()),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['policies'] })
  },
  onError: (err) => toast.error(err.message),
})

// Uso
mutation.mutate({ name: 'Acme' })
// mutation.isPending, isError, isSuccess
```

PATRONES AVANZADOS

Optimistic update

```
useMutation({
  mutationFn: updatePolicy,
  onMutate: async (newData) => {
    await queryClient.cancelQueries({ queryKey: ['policy', id] })
    const previous = queryClient.getQueryData(['policy', id])
    queryClient.setQueryData(['policy', id], newData)
    return { previous }
  },
  onError: (_err, _vars, ctx) => {
    queryClient.setQueryData(['policy', id], ctx?.previous)
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ['policy', id] })
  },
})
```

Prefetch + parallel queries

```
// Al hover en link
queryClient.prefetchQuery({
  queryKey: ['policy', id],
  queryFn: () => fetchPolicy(id),
})

// Varias en paralelo
const results = useQueries({
  queries: ids.map(id => ({
    queryKey: ['policy', id],
    queryFn: () => fetchPolicy(id),
  })),
})
```

USEINFINITEQUERY

```
useInfiniteQuery({
  queryKey: ['policies'],
  queryFn: ({ pageParam = 0 }) =>
    fetchPolicies({ cursor: pageParam }),
  getNextPageParam: (last) => last.nextCursor,
  initialPageParam: 0,
})
// data.pages · fetchNextPage()
// hasNextPage · isFetchingNextPage
```

CUSTOM HOOK (PATRÓN)

```
function usePolicy(id: string) {
  return useQuery({
    queryKey: ['policy', id],
    queryFn: () => api.getPolicy(id),
    enabled: !!id,
  })
}

function useUpdatePolicy() {
  const qc = useQueryClient()
  return useMutation({
    mutationFn: api.updatePolicy,
    onSuccess: (_, { id }) => {
      qc.invalidateQueries({ queryKey: ['policy', id] })
    },
  })
}
```

QUERY VS CONTEXT VS ZUSTAND

Tool	Para
React Query	Datos async del servidor
Context	Theme, locale, auth session
Zustand/Redux	Client state complejo global

Anti-pattern: meter respuestas API en Context → re-renders masivos.

NEXT.JS APP ROUTER

- QueryClientProvider en layout 'use client'
- Prefetch en Server Component → dehydrate/hydrate (SSR)
- Server Components: fetch directo; Client: useQuery
- No duplicar fetch SC + Query sin hidratar

```
// SSR prefetch (avanzado)
const qc = new QueryClient()
await qc.prefetchQuery({ queryKey: ['policies'], queryFn:
  getPolicies })
const state = dehydrate(qc)
// pasar state a HydrationBoundary
```

SIDE EFFECTS — CÓMO LOS MANEJA QUERY

Query **no reemplaza** `useEffect` para todo. Server state → Query/Mutation. UI/DOM → `useEffect`.

Tipo	Herramienta
Fetch + cache + sync	<code>useQuery</code> / <code>useMutation</code>
Efectos tras POST/PUT/DELETE	callbacks de <code>useMutation</code>
Título, analytics, focus	<code>useEffect</code> (si no es del API)
Errores globales de queries	<code>QueryCache</code> <code>onError</code>

`useMutation` — ciclo de vida (patrón principal)

```
useMutation({
  mutationFn: updatePolicy,
  onMutate: async (vars) => {
    // ANTES: optimistic update, cancelQueries
    await queryClient.cancelQueries({ queryKey: ['policy', id] })
    const previous = queryClient.getQueryData(['policy', id])
    queryClient.setQueryData(['policy', id], vars)
    return { previous }
  },
  onSuccess: (data, vars) => {
    // toast, redirect, invalidate, analytics
    queryClient.invalidateQueries({ queryKey: ['policies'] })
  },
  onError: (err, vars, ctx) => {
    // rollback optimistic
    queryClient.setQueryData(['policy', id], ctx?.previous)
    toast.error(err.message)
  },
  onSettled: () => {
    // SIEMPRE al final (ok o error)
  },
})
```

`useQuery` — evitar `onSuccess` (v5)

Mejor derivar en render. Si hace falta efecto en DOM:

```
const { data, isSuccess } = useQuery({ ... })

useEffect(() => {
  if (isSuccess && data) document.title = data.name
}, [isSuccess, data])
```

Global — `QueryCache` / `MutationCache`

```
new QueryClient({
  queryCache: new QueryCache({
    onError: (error, query) => log(query.queryKey, error),
  }),
  mutationCache: new MutationCache({
    onSuccess: () => toast.success('Saved'),
  }),
})
```

`enabled` — evitar fetch prematuro (sin `useEffect`)

```
useQuery({
  queryKey: ['policy', id],
  queryFn: () => fetchPolicy(id),
  enabled: !!id, // no fetch hasta que id exista
})
```

✗ Anti-patterns

```
// ✗ fetch en useEffect
useEffect(() => { fetch('/api').then(setData) }, [])

// ✔ useQuery

// ✗ invalidate en useEffect tras flag manual
useEffect(() => { if (saved) invalidate(...) }, [saved])

// ✔ onSuccess en useMutation
```

Entrevista (EN): “Side effects tied to server ops go in mutation lifecycle hooks; global handlers on QueryCache. I don’t useEffect-fetch — I use enabled and queryKey instead.”

ENTREVISTA — FRASES CLAVE

- “Server state is async, shared, and can go stale — Query owns that.”
- “staleTime vs gcTime: fresh window vs garbage collection.”
- “Invalidation over manual cache sync after mutations.”
- “Optimistic UI with onMutate + rollback onError.”
- “Side effects on mutations: onMutate → onSuccess/onError → onSettled.”
- “queryKey hierarchy enables granular invalidation.”

DEVTOOLS + ERRORES

```
import { ReactQueryDevtools } from
  '@tanstack/react-query-devtools'

<ReactQueryDevtools initialIsOpen={false} />
```

- Global: QueryCache onError para logging
- Retry: solo idempotente (GET), no en POST ciego
- throwOnError + Error Boundary opcional

COMPARAR CON TU SSRM (PLUVIA)

Pluvia ag-Grid SSRM	React Query
Cache por combo filtros	queryKey con filtros
Refetch al cambiar página	queryFn + staleTime
Loading overlay grid	isLoading / isFetching
Invalidate tras save	invalidateQueries

API RÁPIDA QUERYCLIENT

```
queryClient.invalidateQueries({ queryKey: ['policies'] })
queryClient.setQueryData(['policy', id], newData)
queryClient.getQueryData(['policy', id])
queryClient.removeQueries({ queryKey: ['policies'] })
queryClient.resetQueries()
queryClient.cancelQueries({ queryKey: ['policy', id] })
queryClient.prefetchQuery({ queryKey, queryFn })
queryClient.fetchQuery({ queryKey, queryFn }) // await, siempre fetch
```